**TRƯỜNG ĐẠI HỌC SÀI GÒN**
**KHOA CÔNG NGHỆ THÔNG TIN**

# TÀI LIỆU CHUẨN BỊ VẤN ĐÁP

Học phần Nhập môn Dữ liệu lớn

**Đồ án: Olist E-commerce · Analytics → Presentation**

Neon Serverless Postgres + tầng ngữ nghĩa SQL + Apache Superset

Lớp: DCT123C5 – Giảng viên hướng dẫn: Vũ Ngọc Thanh Sang


| STT | Họ và tên | MSSV | Vai trò chính |
|---|---|---|---|
| 1 | Võ Việt Tường | 3123411334 | Nhóm trưởng – kiến trúc, Neon & loader, tích hợp |
| 2 | Nguyễn Hoàng Phi Long | 3123411180 | Semantic layer SQL & dashboard Superset |
| 3 | Mai Hồng Phong | 3123411225 | Thực nghiệm, minh chứng & báo cáo |


Tài liệu nội bộ của nhóm: bảng số liệu, đối chiếu yêu cầu, câu hỏi dự kiến và kịch bản demo.

Thành phố Hồ Chí Minh, năm 2026

# MỤC LỤC

1. Bảng tra nhanh – số liệu cần nhớ ..... 3
2. Trả lời theo từng yêu cầu của đề bài ..... 4
2.1. Mục 1 – Mục tiêu của đồ án ..... 4
2.2. Mục 5.1 – Quy trình được lựa chọn và ranh giới ..... 4
2.3. Mục 5.2 – Bài toán và tính chất Big Data (≥ 2 V) ..... 4
2.4. Mục 5.3 – Input ..... 5
2.5. Mục 5.4 – Output ..... 5
2.6. Mục 5.5 – Công nghệ và lý do chọn ..... 5
2.7. Mục 5.6 – Sơ đồ luồng xử lý ..... 6
2.8. Mục 5.7 – Kết quả demo/thực nghiệm ..... 6
2.9. Mục 5.8 – Khả năng tái lập ..... 6
2.10. Mục 6.2 – Thang điểm và bằng chứng theo từng tiêu chí ..... 7
2.11. Mục 8 – Công cụ AI và tính trung thực học thuật ..... 7
3. Bộ câu hỏi vấn đáp dự kiến và gợi ý trả lời ..... 8
3.1. Tổng quan và pipeline ..... 8
3.2. Vì sao là Big Data? ..... 9
3.3. Neon Postgres ..... 10
3.4. Superset và dashboard ..... 11
3.5. Data contract và kiểm chứng ..... 13
3.6. Hiệu năng và cách đo ..... 14
3.7. Khả năng mở rộng (thí nghiệm ×10) ..... 15
3.8. Hạn chế và trung thực ..... 16
3.9. Gợi ý quyết định, landing page và tái lập ..... 16
3.10. Câu hỏi theo từng thành viên ..... 17
3.11. Câu hỏi "bẫy" ..... 18
4. Kịch bản demo 5–7 phút ..... 20

4.1. Chuẩn bị trước buổi bảo vệ (30 phút trước) ..... 20
4.2. Kịch bản từng bước ..... 20
4.3. Phương án dự phòng ..... 21
4.4. Lưu ý khi trả lời ..... 21

# 1. Bảng tra nhanh – số liệu cần nhớ

Số liệu lấy từ `evidence/` và README tại commit fb9a8b1 (lần nạp 13/11/2026). Khi giảng viên hỏi số, trả lời bằng số tròn trước, số chính xác sau (ví dụ "khoảng 500 nghìn dòng, chính xác 505.592").

**Bảng 1.1.** Số liệu cần nhớ khi vấn đáp

| Hạng mục | Số liệu | Nguồn |
|---|---|---|
| Bộ dữ liệu gốc | Olist Brazil, 99.441 đơn, 09/2016–10/2018; 9 tệp CSV ≈ 1,55 triệu dòng, ~120 MB | Mục 3.1 báo cáo |
| Upstream Spark | Gold: 7 bảng, 33,3 MB, 61,72 s; Analytics: cụm 1 master + 2 worker (4 core), 14 SQL + K-Means k = 4, 119,71 s | `docs/upstream/` |
| Đầu vào của nhóm | 16 bảng Analytics (194.132 dòng, ~8 MB) + 7 bảng gold (311.460 dòng, ~35 MB) = **23 bảng, 505.592 dòng, 42,88 MB Parquet** | `load_run.log` |
| Neon | Project `bigdata-olist-presentation`, PostgreSQL 18.6, **aws-ap-southeast-1 (Singapore)**, **0,25 CU**, gói free, 1 GiB/nhánh, CSDL 146 MB (25 bảng + 22 view) | `neon_project.json`, `neon_tables_inventory.png` |
| Nạp dữ liệu (13/11) | 23/23 bảng khớp; tổng thời gian từng bảng **277,82 s** (log cả overhead 284,74 s); COPY 256,43 s → **1.972 dòng/s**; riêng 16 bảng Analytics 66,44 s; CSV truyền 107,4 MB ≈ 0,42 MB/s | `load_log.csv`, `load_run.log` |
| Kiểm chứng | **23/23** bảng khớp số dòng; **19/19** số neo đạt | `validation.md` |
| Số neo nghiệp vụ | Doanh thu **13.494.400,74 BRL**; đỉnh 11/2017 = 1.003.862 BRL, MoM **+52,06%**; review trễ **2,56** vs đúng hạn **4,29**; khách mua lại **2.887/94.983 = 3,04%**; 539/3.053 người bán (17,65%) tạo 80% doanh thu; 153 người bán watchlist (75 mức ĐỎ) | `validation.md`, README |
| Độ trễ truy vấn | RTT `SELECT 1` **105 ms**; kết nối TLS mới **1.134 ms**; 21 truy vấn dashboard: trung vị **173 ms** phía client, **1,25 ms** phía server; p95 lớn nhất 444 ms | `query_latency.md` |
| Bảng tổng hợp vs tính lại | Pareto danh mục: đọc view **0,24 ms** vs tính lại từ fact **66,43 ms** (server) | `query_latency.md` |
| Tải dashboard (13/11) | Trung vị tab **3,83 s cold → 1,51 s có cache**; **3/7 tab cold > 5 s** (chậm nhất 6,38 s – Khách hàng & RFM); tổng 7 tab 31,38 s → 12,42 s | `dashboard_load_latency.md` |
| Chart qua REST API | 44/44 chart: bỏ cache **1.558 ms** vs cache **67 ms** (~23 lần) | `superset_chart_latency.csv` |
| Bộ lọc | 6/6 kịch bản đúng, 2,0–4,9 s mỗi lần; ví dụ 2018 + SP: doanh thu 13.449.530 → 2.960.326 BRL | `dashboard_load_time.csv`, `filter_*.png` |
| Thí nghiệm ×10 | 112.101 → **1.121.010 dòng** (117,7 MB): truy vấn dữ liệu thô **47 → 507 ms**; lọc có index 2,5 → 18,5 ms; **view tổng hợp giữ 1,1 ms** | `scale_experiment.md` |
| Dashboard | **7 tab, 45 chart, 7 bộ lọc native**, 22 dataset (1 view = 1 dataset); Superset 4.1.2, 4 worker, metadata Postgres 16, cache 600 s | README mục 7 |
| Tầng ngữ nghĩa | 22 view + `dim_state`; `v_decision_hints` 7 luật | `sql/10_presentation_views.sql` |
| Kiểm thử QA | **41/41 PASS** (đăng nhập, 7 tab × 2 độ rộng, 6 bộ lọc, đổi ngôn ngữ, landing, export/import, README) | `qa_test_cases.md` |
| Giao diện | Superset tiếng Việt (bản dịch riêng **536 chuỗi**), thương hiệu Olist Insight; landing page nginx **:8090** | README mục 5 |

# 2. Trả lời theo từng yêu cầu của đề bài

Mỗi mục dưới đây trả lời một yêu cầu trong "Yêu cầu đồ án": nhóm đáp ứng thế nào và bằng chứng nằm ở đâu (tệp trong repo, hình/mục trong báo cáo `report/BaoCao_DoAn_BigData.docx`).

## 2.1. Mục 1 – Mục tiêu của đồ án

**Bảng 2.1.** Đối chiếu năm mục tiêu của đề bài

| Mục tiêu của đề | Nhóm đáp ứng | Bằng chứng |
|---|---|---|
| Nhận diện đặc trưng Big Data, giải thích nhu cầu xử lý phân tán/quy mô lớn | Phân tích Volume, Variety, Veracity, Value; chỉ ra phần phân tán nằm ở Spark (upstream), tầng phục vụ đọc kết quả đã cô đọng; chứng minh bằng thí nghiệm ×10 | Báo cáo 2.1, 3.2, 6.6; `scale_experiment.md` |
| Thiết kế luồng dữ liệu, chỉ rõ thành phần và giao diện dữ liệu | Parquet → loader (COPY, TLS) → Neon (`analytics`, `gold` → `presentation`) → Superset → người dùng / Decision | Hình 4.1, bảng luồng dữ liệu 4.2; `docs/architecture.png` |
| Áp dụng nội dung học phần (Data Lake, Spark, phân tích, trực quan hoá) | Đọc Data Lake Parquet; dùng kết quả Spark SQL + K-Means; serving layer OLAP; dashboard | Báo cáo chương 2, 4 |
| Thí nghiệm/demo có số liệu, log, ảnh, dashboard | 7 kịch bản E1–E7: đối chiếu, nạp, độ trễ, tải dashboard, bộ lọc, ×10, QA 41 ca | Chương 6, 7; thư mục `evidence/` |
| Làm việc nhóm, tài liệu hoá | 3 thành viên, phân công rõ; README, báo cáo, tài liệu này | Phụ lục G; README |

## 2.2. Mục 5.1 – Quy trình được lựa chọn và ranh giới

Nhóm làm **Analytics → Presentation**. *Bắt đầu*: 16 bảng Parquet trong `data/analytics/` (kèm 7 bảng gold) và tài liệu `ANALYTICS_HANDOVER.md`. *Kết thúc*: (1) 25 bảng + 22 view trên Neon; (2) dashboard Superset 7 tab; (3) view `presentation.v_decision_hints` cho nhóm Decision/Action đọc bằng SQL. *Không làm*: không chạy lại Spark, không sửa số của Analytics. Giao diện với nhóm trước là data contract (grain, khoá, kiểu, số neo); với nhóm sau là view SQL ổn định. Bằng chứng: báo cáo 1.2, README mục 1.

## 2.3. Mục 5.2 – Bài toán và tính chất Big Data (≥ 2 V)

Bài toán: đưa kết quả phân tích của một sàn thương mại điện tử tới 5 nhóm người dùng nghiệp vụ (ban điều hành, ngành hàng, logistics, CRM, quản lý người bán) để theo dõi và ra quyết định. Các V nhóm trình bày:

- **Volume:** nguồn ~1,55 triệu dòng; vào tầng của nhóm 505.592 dòng; thí nghiệm nhân bản tới 1,12 triệu dòng trên Neon. Nhóm nói thẳng: khối lượng giảm dần qua

từng tầng — đó là thiết kế đúng của pipeline, phần xử lý phân tán nặng nằm ở Spark.

- **Variety:** 23 bảng, 10+ grain khác nhau (tháng, danh mục, bang, khách, người bán, cụm…), nhiều kiểu (decimal, date, timestamp, boolean), fact phân vùng Hive.
- **Veracity:** cờ chất lượng `dq_*`, tháng thiếu dữ liệu, `customer_id` ≠ khách thật; giữ đúng quy tắc bàn giao và kiểm 19 số neo.
- **Value / phục vụ quy mô lớn:** nhiều người dùng đồng thời, tải không đều → serving layer serverless, cache; luật gợi ý quyết định.

Bằng chứng: báo cáo 3.2, 6.6; README mục 2.

## 2.4. Mục 5.3 – Input

Nguồn: bàn giao của nhóm Analytics (bản sao trong repo `data/analytics/`, `data/gold/`). Định dạng: Parquet + Snappy, mỗi bảng một thư mục kiểu Spark; hai bảng fact phân vùng `purchase_year/purchase_month`. Quy mô: 23 bảng, 505.592 dòng, 42,88 MB. Tốc độ: theo lô (mỗi lần Analytics chạy lại). Trường chính và ví dụ bản ghi: báo cáo 3.3, data contract 3.5, Phụ lục C (schema đầy đủ); `evidence/parquet_schema_inspect.txt`.

## 2.5. Mục 5.4 – Output

(1) Bảng phục vụ trên Neon: schema `analytics` (16 + `load_log`), `gold` (7); (2) tầng ngữ nghĩa `presentation`: 22 view + `dim_state`; (3) dashboard Superset tại `:8088` (7 tab, 45 chart, 7 bộ lọc); (4) `v_decision_hints` 7 luật + `v_seller_watchlist` (mức ĐỎ/VÀNG, lý do, hành động); (5) landing page `:8090` với `web/data/kpi.json` xuất từ Neon; (6) minh chứng trong `evidence/`. Ví dụ đầu ra: một dòng `v_decision_hints` — "ưu tiên 2, CAO, Giao hàng, review đúng hạn 4,29 vs trễ 2,56, ngưỡng chênh > 1 sao, hành động: cảnh báo seller khi đơn trễ 1–3 ngày". Bằng chứng: báo cáo 3.4, 7.3–7.5.

## 2.6. Mục 5.5 – Công nghệ và lý do chọn

**Bảng 2.2.** Công nghệ, vai trò, lý do (tóm tắt để nói)

| Công nghệ | Vai trò | Lý do chọn (một câu) |
|---|---|---|
| Parquet + pyarrow | Đọc input, cả phân vùng Hive | Giữ kiểu Spark (decimal, date), không cần Spark để đọc bảng đã cô đọng |
| Python loader + `COPY` (psycopg 3) trong Docker | Nạp hàng loạt, PK/index, `load_log` | `COPY` nhanh hơn INSERT nhiều lần; chạy lại được; máy Windows không cần cài Python |

| Công nghệ | Vai trò | Lý do chọn (một câu) |
|---|---|---|
| Neon Postgres 18 serverless | Serving layer | SQL chuẩn, tách compute/storage, scale-to-zero, chia sẻ qua Internet + TLS, không phải tự vận hành |
| View SQL | Semantic layer, luật cảnh báo | KPI định nghĩa một nơi, có version control, dùng chung cho dashboard và Decision |
| Apache Superset 4.1.2 | Dashboard, bộ lọc, cache | Mã nguồn mở, REST API dựng dashboard bằng mã (tái lập), Country Map Brazil |
| Postgres 16 (`superset-db`) | Metadata Superset | SQLite khoá ghi toàn cục làm chart tải song song phải xếp hàng |
| Playwright | Ảnh minh chứng, đo tải, QA 41 ca | Tự động, chạy lại được |
| nginx (`web/`) | Landing page `:8090` | Trang tĩnh, chỉ đọc `kpi.json`, không giữ thông tin kết nối |

Bằng chứng: báo cáo 4.6, Phụ lục B (so sánh với Power BI, Tableau, Grafana, Metabase; Postgres cục bộ, ClickHouse, DuckDB).

## 2.7. Mục 5.6 – Sơ đồ luồng xử lý

Sơ đồ data-flow có thành phần, giao thức và nơi xử lý: `docs/architecture.png` (= `evidence/architecture_pipeline.png`), Hình 4.1 báo cáo; bảng 7 bước ở mục 4.2 (Parquet → Arrow/CSV trong RAM → `COPY` qua TLS 5432 endpoint direct → view SQL → SQLAlchemy/psycopg2 endpoint pooled → HTTP 8088 → SQL/CSV/API cho Decision). Bản Mermaid trong `docs/architecture.md`.

## 2.8. Mục 5.7 – Kết quả demo/thực nghiệm

Log: `load_run.log`, `apply_sql.log`; số liệu: `load_log.csv`, `validation.md/json`, `query_latency.md`, `dashboard_load_latency.md`, `superset_chart_latency.csv`, `scale_experiment.md`, `qa_test_cases.md`; ảnh: 7 tab, 6 bộ lọc, watchlist, giao diện tiếng Việt/Olist Insight, landing page, kiểm kê Neon. Báo cáo chương 6 (E1–E7) và chương 7. Kịch bản demo trực tiếp ở mục 4 tài liệu này.

## 2.9. Mục 5.8 – Khả năng tái lập

Toàn bộ chạy bằng Docker Compose (service `superset-db`, `superset`, `tools`, `screenshots`, `web`); dashboard dựng bằng `build_dashboard.py` (hoặc import ZIP); loader và view idempotent; README mục 5 có lệnh từng bước; dữ liệu mẫu (toàn bộ Parquet đầu vào) nằm sẵn trong repo; QA TC41 kiểm mọi lệnh/script README nhắc tới đều tồn tại và `docker compose config` hợp lệ. Bằng chứng: README, Phụ lục A báo cáo, `qa_test_cases.md`.

## 2.10. Mục 6.2 – Thang điểm và bằng chứng theo từng tiêu chí

**Bảng 2.3.** Tự đối chiếu với thang điểm 10

| Tiêu chí (điểm) | Nhóm đáp ứng thế nào | Bằng chứng ở đâu |
|---|---|---|
| Mức độ hoàn thành hệ thống (2,5) | Pipeline chạy đầu-cuối: Parquet → Neon → view → dashboard 7 tab → `v_decision_hints`; 0 chart lỗi; QA 41/41 | Chương 5, 7; `qa_test_cases.md`; ảnh `dashboard_tab*.png` |
| Thiết kế và triển khai kỹ thuật (2,0) | Kiến trúc phân tầng 3 schema; DDL tự sinh; nạp trong giao dịch, idempotent; dashboard-as-code; Docker Compose | Chương 4, 5; `loader/`, `sql/`, `superset/` |
| Thực nghiệm và bằng chứng (2,0) | 7 kịch bản có số đo và cách đo rõ (log, CSV, ảnh); đối chiếu 23/23 + 19/19 | Chương 6; `evidence/` |
| Đặc trưng Big Data và mở rộng (1,5) | Lập luận V's + thí nghiệm ×10 (thô 47 → 507 ms, view 1,1 ms); phân tích nút thắt mạng; hướng mở rộng | Báo cáo 3.2, 6.6, 8.3 |
| Phân tích kết quả và thảo luận (1,0) | Giải thích số (mạng chiếm độ trễ, NullPool, cache); hạn chế trung thực (3/7 tab cold > 5 s) | Chương 6.8, 8 |
| Chất lượng báo cáo và tái lập (1,0) | Báo cáo đúng cấu trúc 6.1, tài liệu tham khảo, README, phân công, ghi chú AI | Báo cáo, README, Phụ lục A, G, H |

## 2.11. Mục 8 – Công cụ AI và tính trung thực học thuật

Nhóm có dùng trợ lý AI (các agent trong Cursor) để gợi ý khung mã (loader, script dựng dashboard, script đo, Dockerfile), gợi ý SQL, dựng script sinh báo cáo và bản nháp nội dung. Nhóm tự chạy toàn bộ hệ thống, đọc và sửa mã, đối chiếu mọi con số với tệp trong `evidence/` và 19 số neo; các quyết định kỹ thuật (Neon, Superset, schema, KPI, luật, tiêu chí đánh giá) do nhóm thảo luận và có lập luận. Đã ghi rõ trong Phụ lục H báo cáo và README mục 10. Khi được hỏi, mỗi thành viên phải giải thích được phần mình phụ trách (xem câu hỏi theo thành viên ở mục 3.10).

# 3. Bộ câu hỏi vấn đáp dự kiến và gợi ý trả lời

Mỗi câu ghi kèm người nên trả lời chính (trong ngoặc). Câu trả lời viết theo văn nói, xưng "nhóm em"; nên nói ý chính trước, số liệu sau, và chỉ vào bằng chứng (ảnh, tệp) khi có thể.

## 3.1. Tổng quan và pipeline

***Câu 1 (Tường). Nhóm em làm phần nào trong pipeline, bắt đầu và kết thúc ở đâu?***

**Trả lời:** Dạ, nhóm em làm quy trình con Analytics → Presentation. Đầu vào là 16 bảng Parquet do nhóm Analytics tính trên Spark, kèm 7 bảng gold để lọc chéo, tổng 23 bảng, khoảng 505 nghìn dòng. Nhóm em nạp vào Neon Postgres, xây tầng ngữ nghĩa bằng 22 view SQL và trình bày trên dashboard Superset 7 tab, 45 biểu đồ, 7 bộ lọc. Điểm kết thúc là dashboard cho người dùng nghiệp vụ và view `v_decision_hints` để nhóm Decision/Action đọc bằng SQL. Nhóm em không chạy lại Spark và không sửa số của nhóm Analytics.

***Câu 2 (Tường). Tóm tắt kiến trúc hệ thống trong 30 giây?***

**Trả lời:** Dạ, có bốn khối. Một là Data Lake Parquet. Hai là loader Python chạy trong Docker: đọc Parquet bằng pyarrow, nạp bằng lệnh COPY qua TLS vào Neon. Ba là Neon có ba schema: `analytics` và `gold` là bản sao vật lý, `presentation` là tầng ngữ nghĩa. Bốn là Superset kết nối Neon qua SQLAlchemy, mỗi view là một dataset, dashboard được dựng bằng REST API. Sơ đồ ở `docs/architecture.png`.

***Câu 3 (Phi Long). Dashboard trả lời những câu hỏi nghiệp vụ nào?***

**Trả lời:** Dạ, mỗi tab dành cho một nhóm người dùng. Tab Tổng quan cho ban điều hành: doanh thu tăng thế nào, vận hành có theo kịp không. Tab Danh mục: ngành nào tạo 80% doanh thu, mùa vụ ra sao. Tab Địa lý: bang nào giao chậm, review thấp. Tab Khách hàng: bao nhiêu khách quay lại, nên chăm sóc phân khúc nào. Tab Người bán: ai lớn nhưng vận hành kém. Tab Giao trễ: trễ bao nhiêu ngày thì review sụt. Tab cuối là gợi ý quyết định và nguồn dữ liệu.

***Câu 4 (Hồng Phong). Kết quả phân tích nổi bật nhất dashboard cho thấy là gì?***

**Trả lời:** Dạ, có ba ý. Thứ nhất, bán nhiều không đồng nghĩa vận hành tốt: tháng 11/2017 doanh thu đỉnh 1,0 triệu BRL, tăng 52% so với tháng trước, nhưng tỷ lệ trễ

lên 14,3% và review rơi còn 3,95. Thứ hai, giao trễ làm review giảm từ 4,29 xuống 2,56 sao, trễ từ 4 ngày trở lên thì đa số khách cho 1 sao. Thứ ba, 97% khách chỉ mua một lần, chỉ 3,04% quay lại, nên CRM phải tập trung vào nhóm nhỏ đó.

## 3.2. Vì sao là Big Data?

***Câu 5 (Cả nhóm). Tầng của nhóm chỉ có khoảng 500 nghìn dòng, vậy đây có phải Big Data thật không?***

**Trả lời:** Dạ, nhóm em xin trả lời thẳng là ở tầng Presentation dữ liệu không còn lớn, và đó là thiết kế đúng của pipeline: khối lượng giảm dần qua từng tầng, từ khoảng 1,55 triệu dòng thô xuống các bảng đã tổng hợp. Phần xử lý phân tán nặng nằm ở Spark của nhóm trước, nhóm em có ảnh Spark UI trong báo cáo. Bài toán của tầng phục vụ là khác: nhiều người dùng đồng thời, độ trễ tương tác, tải không đều, giữ đúng ngữ nghĩa số liệu. Nhóm em chứng minh khả năng mở rộng bằng thí nghiệm nhân dữ liệu lên 10 lần: truy vấn trên dữ liệu thô tăng từ 47 lên 507 ms, còn view đã tổng hợp giữ 1,1 ms. Nghĩa là kiến trúc giữ nguyên khi dữ liệu nguồn lớn lên.

***Câu 6 (Hồng Phong). Bài toán thể hiện những đặc trưng V nào?***

**Trả lời:** Dạ, nhóm em chọn bốn. Volume: nguồn 1,55 triệu dòng, và thí nghiệm đã chạy tới 1,12 triệu dòng trên Neon. Variety: 23 bảng với hơn 10 grain khác nhau, nhiều kiểu dữ liệu, có fact phân vùng theo tháng. Veracity: có cờ chất lượng `dq_*`, tháng thiếu dữ liệu, `customer_id` không phải khách thật, nên nhóm em kiểm 19 số neo. Value: kết quả thành luật quyết định, ví dụ 75 người bán mức cảnh báo đỏ.

***Câu 7 (Tường). Tại sao không dùng Spark cho tầng Presentation?***

**Trả lời:** Dạ, vì Spark là hệ xử lý theo lô, mỗi job tốn hàng chục giây để khởi động và lập kế hoạch, không hợp để trả lời từng cú nhấp của người dùng trong vài trăm mili giây. Phần tính nặng đã được Spark làm xong ở nhóm Analytics; tầng của nhóm em chỉ cần đọc, lọc và cộng lại trên bảng đã cô đọng, việc mà Postgres có chỉ mục làm trong khoảng 1 ms phía server. Đây đúng là mô hình batch layer và serving layer trong kiến trúc Lambda. Ngoài ra, đọc Parquet bằng pyarrow trong loader cũng không cần cụm Spark.

***Câu 8 (Cả nhóm). Nếu dữ liệu tăng 100 lần thì sao?***

**Trả lời:** Dạ, các bảng theo tháng, danh mục, bang gần như không đổi vì số tháng và số danh mục cố định; chỉ bảng cấp khách hàng, người bán và gold tăng tuyến tính. Thí nghiệm ×10 cho thấy truy vấn thô tăng gần tuyến tính còn view tổng hợp không đổi, nên ở ×100 nhóm em sẽ không cho dashboard truy vấn fact thô nữa mà chuyển lớp gold thành bảng tổng hợp nhẹ hoặc materialized view. Neon có thể tăng compute và thêm read replica; Superset mở rộng ngang với Celery và Redis. Nếu cần lọc chéo tự do trên hàng tỷ dòng chi tiết thì chuyển tầng phục vụ sang kho OLAP cột như ClickHouse hoặc Trino; vì tầng ngữ nghĩa là SQL chuẩn nên chủ yếu chỉ phải chuyển view, dashboard giữ nguyên.

## 3.3. Neon Postgres

***Câu 9 (Tường). Neon "serverless" nghĩa là gì, khác Postgres thường thế nào?***

**Trả lời:** Dạ, Neon tách compute và storage. Compute là tiến trình Postgres không giữ dữ liệu, có thể tự tăng giảm hoặc tắt hẳn khi không dùng; storage gồm safekeeper nhận WAL, pageserver dựng lại trang dữ liệu và object storage lưu lâu dài. Vì vậy dữ liệu luôn an toàn dù compute ngủ. Với dashboard, tải rất không đều nên không ai xem thì không tốn compute. Project của nhóm em ở vùng Singapore, PostgreSQL 18, gói miễn phí 0,25 CU.

***Câu 10 (Tường). Endpoint pooled và direct khác nhau thế nào, nhóm dùng cái nào?***

**Trả lời:** Dạ, endpoint pooled có chữ `-pooler` trong hostname, đi qua PgBouncer chế độ transaction, chịu được rất nhiều kết nối ngắn nên Superset và script đo độ trễ dùng nó. Endpoint direct nối thẳng Postgres, giữ phiên ổn định, phù hợp cho DDL và COPY dài; loader tự bỏ `-pooler` để dùng direct. Nếu cho COPY đi qua PgBouncer chế độ transaction thì dễ gặp lỗi vì trạng thái phiên không được giữ.

***Câu 11 (Tường). Scale-to-zero ảnh hưởng gì đến người dùng? Có đo cold start không?***

**Trả lời:** Dạ, khi không có truy vấn một thời gian, compute tạm dừng, truy vấn đầu tiên phải chờ đánh thức. Nhóm em chưa đo tách riêng thời gian đánh thức compute; tài liệu Neon cho biết cỡ vài trăm mili giây. Cái nhóm em đo được là thời gian mở dashboard khi đã xoá cache: trung vị 3,83 giây một tab, trong đó phần lớn do

mỗi biểu đồ mở kết nối TLS mới mất khoảng 1,1 giây, chứ không phải do compute. Có cache thì còn 1,51 giây.

***Câu 12 (Cả nhóm). Sao không dùng Postgres cài trên máy, ClickHouse hay BigQuery?***

**Trả lời:** Dạ, Postgres trên máy thì nhanh nhưng dữ liệu nằm trên laptop một bạn, không chia sẻ được cho nhóm, thầy và nhóm Decision. ClickHouse mạnh với hàng tỷ dòng chi tiết, nhưng với nửa triệu dòng đã tổng hợp thì không thấy lợi thế mà lại thêm việc vận hành. BigQuery hay Snowflake tính phí theo truy vấn, cần thẻ thanh toán và khoá vào một nhà cung cấp. Neon cho Postgres chuẩn, miễn phí ở quy mô đồ án, có TLS, mọi công cụ đều kết nối được. Bảng so sánh ở Phụ lục B báo cáo.

***Câu 13 (Tường). Loader nạp dữ liệu thế nào, chạy lại có bị trùng không?***

**Trả lời:** Dạ, loader đọc mỗi bảng bằng `pyarrow.dataset`, đọc được cả thư mục phân vùng Hive, tự sinh câu `CREATE TABLE` từ schema Parquet, chuyển thành CSV trong RAM rồi `COPY FROM STDIN` theo khối 1 MiB. DROP, CREATE và COPY nằm trong một giao dịch nên người dùng chỉ thấy bản cũ hoặc bản mới hoàn chỉnh. Sau đó tạo khoá chính, chỉ mục, ANALYZE, đếm dòng so với Parquet và ghi `analytics.load_log`. Chạy lại bao nhiêu lần cũng ra cùng trạng thái, không trùng dữ liệu.

***Câu 14 (Tường). Vì sao tiền dùng NUMERIC chứ không dùng float?***

**Trả lời:** Dạ, vì số neo doanh thu 13.494.400,74 BRL phải khớp đến từng xu. Kiểu số thực dấu phẩy động cộng hàng chục nghìn giá trị sẽ có sai số làm tròn. Parquet lưu decimal nên nhóm em ánh xạ sang `NUMERIC(p,s)`; kiểm tra số neo cho thấy tổng trên Neon bằng đúng số của Spark.

## 3.4. Superset và dashboard

***Câu 15 (Phi Long). Vì sao chọn Superset mà không dùng Power BI, Tableau, Grafana hay Metabase?***

**Trả lời:** Dạ, Power BI và Tableau là phần mềm thương mại, tệp dashboard nhị phân khó quản lý phiên bản, chia sẻ cần tài khoản trả phí, Power BI Desktop chỉ chạy trên Windows. Grafana mạnh về chuỗi thời gian giám sát nhưng yếu với Pareto, bảng xếp hạng. Metabase dễ dùng nhưng ít loại biểu đồ và bộ lọc hạn chế. Superset mã

nguồn mở, chạy bằng Docker, kết nối thẳng Postgres, có bản đồ Brazil, và quan trọng nhất là có REST API để dựng dashboard bằng mã, giúp người khác dựng lại giống hệt.

***Câu 16 (Phi Long). Tầng ngữ nghĩa là gì, sao không để Superset tự tính?***

**Trả lời:** Dạ, tầng ngữ nghĩa là nơi định nghĩa KPI một lần, ở đây là 22 view trong schema `presentation`. Nếu mỗi biểu đồ tự viết SQL thì dễ ra mỗi nơi một số. Đặt trong view thì Superset và nhóm Decision đọc cùng một định nghĩa, có quản lý phiên bản bằng Git. View cũng thêm cột phục vụ trình bày như `month_date` cho bộ lọc thời gian, mã ISO `BR-SP` cho bản đồ, nhãn tiếng Việt, và mã hoá luật cảnh báo.

***Câu 17 (Phi Long). Tỷ lệ trễ khi lọc nhiều tháng tính thế nào cho đúng?***

**Trả lời:** Dạ, đây là độ đo không cộng dồn. Bảng `kpi_monthly` chỉ có tỷ lệ theo tháng, nếu lấy trung bình các tỷ lệ thì sai. Nhóm em làm view cấp đơn `v_orders` từ gold, cột `late_flag` bằng 1 khi trễ, 0 khi đúng hạn, NULL khi chưa giao; `AVG(late_flag)` chính là tỷ lệ trễ đúng định nghĩa với mọi tổ hợp bộ lọc. AOV cũng tính bằng tổng giá trị đơn chia số đơn chứ không lấy trung bình của AOV tháng.

***Câu 18 (Phi Long). Bộ lọc native hoạt động thế nào, có lọc được mọi biểu đồ không?***

**Trả lời:** Dạ, có 7 bộ lọc: khoảng thời gian, bang khách hàng, vùng, danh mục, phân khúc RFM, phương thức thanh toán, bang người bán. Mỗi bộ lọc chỉ gắn vào biểu đồ có cột tương ứng, phạm vi do script tính tự động. Các kết quả phân tích toàn kỳ như Pareto hay K-Means thì không lọc theo tháng được vì đó là kết quả Spark đã chốt, lọc lại sẽ sai nghĩa. Nhóm em kiểm 6 kịch bản lọc, cả 6 đúng; ví dụ lọc năm 2018 và bang SP thì doanh thu từ 13,45 triệu còn 2,96 triệu BRL.

***Câu 19 (Phi Long). Cache của Superset để làm gì? Metadata lưu ở đâu?***

**Trả lời:** Dạ, Superset lưu kết quả truy vấn 600 giây, lần mở lại không phải hỏi Neon: một biểu đồ từ 1.558 ms còn 67 ms, nhanh khoảng 23 lần. Metadata gồm người dùng, biểu đồ, dashboard lưu trong container Postgres 16 riêng. Ban đầu nhóm em dùng SQLite nhưng SQLite khoá ghi toàn cục làm các biểu đồ tải song song phải xếp hàng, nên chuyển sang Postgres và chạy 4 worker.

***Câu 20 (Phi Long). Nếu xoá container Superset thì dashboard có mất không?***

**Trả lời:** Dạ, không mất vì dashboard được dựng bằng mã. Chạy `build_dashboard.py` là tạo lại database, 22 dataset, 45 biểu đồ, 7 tab và 7 bộ lọc; script idempotent nên chạy lại chỉ cập nhật. Dự phòng thêm có tệp ZIP export và `import_dashboard.py`. Dữ liệu nghiệp vụ nằm trên Neon nên cũng không bị ảnh hưởng.

***Câu 21 (Phi Long). Giao diện tiếng Việt và theme Olist Insight làm thế nào?***

**Trả lời:** Dạ, Superset 4.1.2 không có sẵn tiếng Việt nên nhóm em viết bản dịch riêng khoảng 536 chuỗi hay gặp, biên dịch khi build image. Phiên mới mặc định tiếng Việt, vẫn đổi được sang English bằng cờ, và trang tải lại đúng tab, bộ lọc đang mở. Theme dùng logo, màu chàm và xanh ngọc, font Be Vietnam Pro đồng bộ với landing page. Giới hạn là một số màn hình như SQL Lab và màn hình sửa biểu đồ còn tiếng Anh, và Superset 4.1 chưa có dark mode thật cho biểu đồ.

## 3.5. Data contract và kiểm chứng

***Câu 22 (Tường). Data contract giữa hai nhóm gồm những gì?***

**Trả lời:** Dạ, gồm danh sách bảng, grain tức một dòng là gì, khoá, kiểu dữ liệu, ngữ nghĩa từng độ đo, khung thời gian 2017-01 đến 2018-08, và các số neo để đối chiếu. Thêm cột thì loader tự xử lý; đổi tên hay xoá cột là thay đổi phá vỡ phải báo trước. Mỗi bảng trên Neon có COMMENT ghi grain và nguồn. Chi tiết ở mục 3.5 và Phụ lục C báo cáo.

***Câu 23 (Hồng Phong). Làm sao chứng minh số trên dashboard đúng?***

**Trả lời:** Dạ, script `validate.py` kiểm hai lớp. Một là đọc lại từng thư mục Parquet và so với `COUNT(*)` trên Postgres: 23/23 bảng khớp. Hai là chạy 19 truy vấn số neo: tổng doanh thu 13.494.400,74 BRL, tháng đỉnh 2017-11 tăng 52,06%, review 2,56 và 4,29, 2.887 khách mua lại, 153 người bán watchlist — 19/19 đạt. Có cả kiểm tra nhất quán chéo: tổng từ lớp gold bằng đúng tổng của bảng Analytics, chênh lệch 0,00. QA còn đọc trực tiếp dữ liệu trả về của biểu đồ sau khi lọc.

***Câu 24 (Hồng Phong). Sao tổng doanh thu trên thẻ KPI là 13,45 triệu mà số neo là 13,49 triệu?***

**Trả lời:** Dạ, cả hai đều đúng nhưng trả lời hai câu hỏi khác nhau. 13.494.400,74 BRL là doanh thu gold toàn kỳ, kể cả các tháng 2016 và 09/2018 thiếu dữ liệu. Thẻ KPI dùng khung chuẩn 2017-01 đến 2018-08 theo quy tắc bàn giao nên ra 13.449.530 BRL. Thẻ có ghi rõ khung thời gian để người dùng không hiểu nhầm.

***Câu 25 (Phi Long). Nạp thêm 7 bảng gold có làm "hai sự thật" không?***

**Trả lời:** Dạ, nhóm em lo điều đó nên chỉ dùng gold để lọc và cộng lại, còn mọi phân tích như Pareto, RFM, K-Means vẫn lấy nguyên của Analytics. Kiểm tra nhất quán chéo cho thấy tổng doanh thu và số đơn từ `v_orders`, `v_sales_items` bằng đúng tổng của `kpi_monthly`. Nên không có hai con số mâu thuẫn.

## 3.6. Hiệu năng và cách đo

***Câu 26 (Hồng Phong). Nạp dữ liệu mất bao lâu? Nút thắt ở đâu?***

**Trả lời:** Dạ, lần nạp ngày 13/11 nạp 23 bảng, 505.592 dòng, tổng thời gian các bảng 277,8 giây, log tính cả chi phí khởi tạo là 284,7 giây; riêng 16 bảng Analytics mất 66 giây. Pha COPY đạt khoảng 1.972 dòng mỗi giây. Nút thắt là băng thông tải lên từ máy nhà tới Singapore, khoảng 0,42 MB/s, vì CSV gửi đi lớn hơn Parquet khoảng 2,5 lần; ngoài ra mỗi bảng nhỏ tốn khoảng 1–2 giây cho nhiều vòng khứ hồi mạng. Muốn nhanh thì chạy loader cùng vùng với Neon.

***Câu 27 (Hồng Phong). Độ trễ truy vấn đo thế nào, kết quả ra sao?***

**Trả lời:** Dạ, script `measure_latency.py` chạy đúng 21 truy vấn mà các biểu đồ sinh ra, qua endpoint pooled giống Superset, mỗi truy vấn 5 lần sau một lần làm nóng, đo cả phía client và thời gian thực thi phía server bằng EXPLAIN ANALYZE. Trung vị phía client 173 ms nhưng phía server chỉ 1,25 ms; riêng một vòng khứ hồi `SELECT 1` đã 105 ms. Kết luận là mạng chiếm gần hết độ trễ, không phải cơ sở dữ liệu.

***Câu 28 (Hồng Phong). Mở dashboard mất bao lâu? Vì sao có tab trên 5 giây?***

**Trả lời:** Dạ, đo bằng Playwright ngày 13/11: xoá cache rồi mở 7 tab, trung vị 3,83 giây một tab; mở lại khi có cache trung vị 1,51 giây. Có 3/7 tab vượt 5 giây khi cold, chậm nhất là tab Khách hàng & RFM 6,38 giây. Lý do là Superset dùng NullPool, mỗi biểu đồ mở kết nối TLS mới tới Singapore khoảng 1,1 giây và các biểu đồ xếp

hàng chờ, chứ truy vấn chỉ khoảng 1 ms. Nhóm em ghi rõ tiêu chí này chưa đạt và đề xuất bật pool kết nối, đặt Superset cùng vùng hoặc làm nóng cache định kỳ.

***Câu 29 (Hồng Phong). Tại sao đo cache nhanh hơn 23 lần mà tab chỉ nhanh khoảng 2,5 lần?***

**Trả lời:** Dạ, con số 23 lần là cho từng biểu đồ qua API: 1.558 ms xuống 67 ms. Thời gian mở một tab còn gồm trình duyệt vẽ biểu đồ ECharts, tải khung trang và chờ biểu đồ chậm nhất trong tab, các phần này cache không rút ngắn được. Vì vậy tab giảm từ 3,83 xuống 1,51 giây.

## 3.7. Khả năng mở rộng (thí nghiệm ×10)

***Câu 30 (Hồng Phong). Thí nghiệm mở rộng làm thế nào?***

**Trả lời:** Dạ, script `scale_experiment.py` nhân bản bảng `gold.fact_order_items` lên 1, 2, 5, 10 lần ngay trên Neon bằng `generate_series`, tới 1,12 triệu dòng, 117,7 MB; sinh dữ liệu khoảng 1 triệu dòng mỗi giây phía server. Với mỗi hệ số, đo ba kiểu truy vấn bằng EXPLAIN ANALYZE, lấy trung vị 3 lần. Xong thì xoá bảng tạm để không vượt giới hạn 1 GiB của gói miễn phí.

***Câu 31 (Hồng Phong). Kết quả thí nghiệm nói lên điều gì?***

**Trả lời:** Dạ, truy vấn tổng hợp tháng × danh mục trên dữ liệu thô tăng gần tuyến tính từ 47 lên 507 ms. Truy vấn có lọc dùng chỉ mục tăng chậm hơn nhiều, từ 2,5 lên 18,5 ms. Còn đọc view đã tổng hợp ở Spark giữ 1,1 ms ở mọi hệ số. Điều này chứng minh thiết kế: tầng Presentation đọc kết quả đã cô đọng nên độ trễ dashboard không tăng theo dữ liệu nguồn; phần tăng được đẩy về Spark vốn mở rộng ngang được.

***Câu 32 (Tường). Thí nghiệm chỉ tới ×10, vậy có đủ thuyết phục không?***

**Trả lời:** Dạ, nhóm em dừng ở ×10 vì gói miễn phí giới hạn 1 GiB mỗi nhánh và compute 0,25 CU. Xu hướng tuyến tính đã rõ qua bốn điểm đo nên có thể ngoại suy, nhưng nhóm em không khẳng định con số chính xác ở ×100. Điều nhóm em khẳng định là đường của view tổng hợp nằm ngang. Với tải thật lớn hơn, cần nâng compute và kiểm chứng lại.

## 3.8. Hạn chế và trung thực

***Câu 33 (Cả nhóm). Hạn chế lớn nhất của hệ thống là gì?***

**Trả lời:** Dạ, có bốn điểm. Một, mở dashboard lần đầu chậm: 3/7 tab cold trên 5 giây do Superset mở kết nối mới cho mỗi biểu đồ. Hai, phụ thuộc mạng Việt Nam – Singapore, cả nạp lẫn truy vấn. Ba, chỉ chạy theo lô, nạp lại toàn bộ, chưa có streaming hay nạp tăng dần. Bốn, gói Neon miễn phí 0,25 CU nên chưa kiểm chứng autoscaling và tải hàng nghìn người dùng.

***Câu 34 (Tường). Có ảnh chụp Neon Console không?***

**Trả lời:** Dạ, không ạ. Neon Console cần đăng nhập tương tác nên nhóm em không chụp tự động được. Thay vào đó, ảnh kiểm kê được sinh từ truy vấn catalog trực tiếp trên database: 25 bảng, 22 view, số dòng, dung lượng 146 MB; thông tin project như vùng, phiên bản, 0,25 CU lấy qua Neon API và lưu ở `neon_project.json`.

***Câu 35 (Phi Long). Bảng giá trên landing page có thật không?***

**Trả lời:** Dạ, không ạ, đó là bảng giá minh hoạ cho kịch bản thương mại hoá, trên trang có ghi rõ. Bản đồ án chạy hoàn toàn trên gói miễn phí của Neon và phần mềm mã nguồn mở.

***Câu 36 (Phi Long). Superset có chỗ nào chưa Việt hoá hoặc chưa đẹp?***

**Trả lời:** Dạ có. Bản dịch 536 chuỗi phủ các màn hình hay dùng, nhưng SQL Lab, màn hình sửa biểu đồ và trang quản trị còn nhiều chữ tiếng Anh; các giá trị dữ liệu như tên danh mục hay "Champions", "At Risk" giữ nguyên gốc. Superset 4.1 chưa có dark mode thật cho biểu đồ, nên thẻ biểu đồ để nền sáng cho dễ đọc. Đây là giới hạn của phiên bản, nhóm em ghi trong README.

## 3.9. Gợi ý quyết định, landing page và tái lập

***Câu 37 (Phi Long). "Gợi ý quyết định" gồm những gì, nhóm Decision dùng thế nào?***

**Trả lời:** Dạ, view `v_decision_hints` có 7 luật, mỗi dòng gồm mức ưu tiên, mức độ, tín hiệu, giá trị hiện tại, ngưỡng và hành động đề xuất. Ví dụ luật 1: 75/153 người bán top doanh thu vi phạm từ hai tiêu chí trở lên thì cảnh cáo, giảm hiển thị, yêu cầu kế hoạch SLA 30 ngày; luật 2: đơn trễ kéo review từ 4,29 xuống 2,56 thì cảnh báo người bán khi đơn trễ 1–3 ngày. Nhóm Decision chỉ cần một câu SELECT để lấy, rồi

chuyển thành email hay Telegram. Giá trị tính lại mỗi lần truy vấn nên luật tự cập nhật khi nạp dữ liệu mới.

***Câu 38 (Hồng Phong). Landing page có vai trò gì trong đồ án?***

**Trả lời:** Dạ, landing page ở cổng 8090 là trang giới thiệu hệ thống cho người xem lần đầu: biểu đồ phát lại doanh thu 20 tháng, thẻ KPI, insight, pipeline và huy hiệu kiểm chứng 19/19 số neo. Số liệu là số thật xuất từ Neon bằng `export_landing_kpi.py` thành `kpi.json`; trang tĩnh nên không giữ thông tin kết nối cơ sở dữ liệu. Logo trên Superset dẫn về landing và nút trên landing mở đúng dashboard. Nó không thay thế dashboard, chỉ là cửa vào.

***Câu 39 (Tường). Người khác muốn chạy lại hệ thống thì làm thế nào?***

**Trả lời:** Dạ, chỉ cần Docker, Git và một database Neon. Clone repo, chép `.env.example` thành `.env` và điền chuỗi kết nối; rồi chạy `docker compose run --rm tools python loader/load_to_neon.py` để nạp, `validate.py` để kiểm, `docker compose up -d superset-db superset` và `build_dashboard.py` để dựng dashboard. Dữ liệu Parquet đã có sẵn trong repo. QA đã kiểm mọi lệnh trong README đều chạy được và `docker compose config` hợp lệ.

***Câu 40 (Hồng Phong). Kiểm thử QA gồm những gì?***

**Trả lời:** Dạ, script `qa_check.py` dùng Playwright chạy 41 ca tự động: đăng nhập đúng, sai mật khẩu; 7 tab ở hai độ rộng 1440 và 1280 px kiểm biểu đồ lỗi, lỗi console, thẻ KPI chồng chữ; 6 kịch bản lọc kiểm giá trị trả về; xoá lọc; đổi ngôn ngữ giữ nguyên trang; link logo và landing; xuất nhập ZIP; lệnh README. Kết quả 41/41 PASS, chi tiết trong `qa_test_cases.md`.

## 3.10. Câu hỏi theo từng thành viên

***Câu 41 (Tường). Em phụ trách phần nào, khó nhất là gì?***

**Trả lời:** Dạ, em là nhóm trưởng, phụ trách kiến trúc, Neon, loader, Docker Compose và tích hợp, quản lý repo GitHub. Khó nhất là nạp đúng kiểu dữ liệu: tiền phải là NUMERIC, timestamp phải ép từ nano về micro giây, chuỗi rỗng phải phân biệt với NULL, và bảng fact phân vùng phải đọc được cột năm, tháng từ tên thư mục. Em xử lý bằng cách sinh DDL từ schema Parquet và kiểm 23/23 số dòng sau mỗi lần nạp.

***Câu 42 (Tường). Nếu loader chết giữa chừng thì dữ liệu có hỏng không?***

**Trả lời:** Dạ, không ạ. Mỗi bảng nạp trong một giao dịch, lỗi thì rollback, bảng cũ vẫn nguyên. Chạy lại loader thì nạp lại từ đầu và ra cùng trạng thái. Nếu số dòng lệch, chương trình thoát với mã lỗi để không âm thầm đi tiếp.

***Câu 43 (Phi Long). Em phụ trách phần nào? Cho ví dụ một view em viết.***

**Trả lời:** Dạ, em phụ trách tầng ngữ nghĩa và dashboard. Ví dụ `v_seller_watchlist`: mỗi tiêu chí review dưới 4, giao trên 15 ngày, trễ trên 10% thành một cờ 0/1; cộng lại thành số vi phạm; ghép thành chuỗi lý do; từ hai vi phạm trở lên là mức ĐỎ kèm hành động "cảnh cáo, giảm hiển thị". Kết quả có 75 người bán mức ĐỎ trên 153. Dashboard thì em dựng bằng `build_dashboard.py` qua REST API, gồm 7 tab, 45 biểu đồ và 7 bộ lọc.

***Câu 44 (Phi Long). Vì sao bản đồ Brazil cần bảng `dim_state`?***

**Trả lời:** Dạ, biểu đồ Country Map của Superset nhận diện bang theo mã ISO 3166-2 dạng `BR-SP`, còn dữ liệu chỉ có `SP`. Bảng `dim_state` 27 dòng thêm tên bang, vùng và mã ISO; nhờ cột vùng mới có bộ lọc Vùng, ví dụ lọc Nordeste còn đúng 9 bang.

***Câu 45 (Hồng Phong). Em phụ trách phần nào? Số liệu em đo có tin được không?***

**Trả lời:** Dạ, em phụ trách thực nghiệm, minh chứng, QA và báo cáo. Mọi số đo đều do script sinh ra và lưu thành tệp trong `evidence/`, có ghi thời điểm đo, cách đo, số lần lặp; người khác chạy lại cùng lệnh sẽ ra số cùng bậc, chỉ dao động theo đường truyền. Ví dụ lần đo trước tab cold trung vị 6,64 giây, lần đo 13/11 là 3,83 giây; em ghi cả hai và giải thích dao động chủ yếu do mạng.

## 3.11. Câu hỏi "bẫy"

***Câu 46 (Cả nhóm). AI làm hết phải không?***

**Trả lời:** Dạ, không ạ. Nhóm em có dùng trợ lý AI trong Cursor để gợi ý khung mã, SQL và bản nháp báo cáo, và đã ghi rõ trong Phụ lục H. Nhưng nhóm em tự chạy toàn bộ hệ thống, sửa lỗi thực tế như SQLite khoá ghi, kết nối pooled và direct, đối chiếu mọi số với tệp trong `evidence/`. Các quyết định chọn Neon, Superset, thiết kế schema, định nghĩa KPI và luật là của nhóm. Thầy có thể hỏi bất kỳ phần nào, mỗi bạn giải thích được phần mình làm.

***Câu 47 (Cả nhóm). Dashboard này khác gì làm bằng Excel?***

**Trả lời:** Dạ, khác ở ba điểm. Dữ liệu nằm trên cơ sở dữ liệu dùng chung, nhiều người truy cập đồng thời và luôn cùng một phiên bản số. Định nghĩa KPI nằm trong view có kiểm chứng tự động, không phụ thuộc thao tác tay. Và toàn bộ dựng lại được bằng script, mở rộng được khi dữ liệu lớn lên, điều mà Excel không làm được ở quy mô hàng trăm nghìn tới hàng triệu dòng.

***Câu 48 (Tường). Có xử lý thời gian thực (streaming) không?***

**Trả lời:** Dạ, không ạ. Dữ liệu Olist là dữ liệu lịch sử và nhóm Analytics bàn giao theo lô, nên hệ thống của nhóm em là batch: mỗi lần Analytics chạy lại thì nạp lại. Loader idempotent nên có thể chạy theo lịch, ví dụ mỗi giờ. Nếu cần gần thời gian thực thì phải có nguồn streaming ở các tầng trước và nạp tăng dần, đó là hướng phát triển.

***Câu 49 (Tường). Dữ liệu trên Neon có an toàn không, mật khẩu để ở đâu?***

**Trả lời:** Dạ, kết nối bắt buộc TLS với `sslmode=require`. Chuỗi kết nối và mật khẩu chỉ nằm trong tệp `.env`, tệp này nằm trong `.gitignore` nên không lên GitHub; repo chỉ có `.env.example`. Loader che mật khẩu khi ghi log, và landing page chỉ đọc tệp `kpi.json` tĩnh nên không lộ thông tin kết nối.

***Câu 50 (Hồng Phong). Nếu Neon hoặc Docker hỏng lúc bảo vệ thì sao?***

**Trả lời:** Dạ, nhóm em đã chuẩn bị phương án dự phòng: toàn bộ ảnh chụp 7 tab, 6 bộ lọc, watchlist và landing page có trong `evidence/`, cùng bản PDF báo cáo và các tệp số liệu. Nhóm em sẽ trình bày bằng ảnh và log, và có thể chạy lại phần kiểm chứng sau nếu thầy cần.

***Câu 51 (Phi Long). Tại sao 97% khách chỉ mua một lần mà vẫn làm phân khúc RFM?***

**Trả lời:** Dạ, chính vì vậy nên nhóm Analytics tách luật: chỉ khách mua từ hai lần mới được gọi Champions, Loyal hay At Risk, khách mua một lần chia theo độ mới. Dashboard cho thấy CRM nên tập trung vào 2.887 khách mua lại, đặc biệt 1.961 khách At Risk, thay vì gửi khuyến mãi cho 65 nghìn khách mua một lần đã lâu. K-Means cũng tự tách đúng 2.887 khách mua lại thành một cụm, xác nhận tần suất là trục phân biệt thật.

# 4. Kịch bản demo 5–7 phút

## 4.1. Chuẩn bị trước buổi bảo vệ (30 phút trước)

1. Bật Docker Desktop, chạy `docker compose up -d superset-db superset web` và chờ trạng thái `healthy` (`docker compose ps`).
2. Đánh thức Neon và làm nóng cache: đăng nhập `http://localhost:8088`, mở dashboard `olist-presentation`, bấm lần lượt 7 tab (lần mở đầu tab chậm 3–6 s, các lần sau ~1,5 s).
3. Mở sẵn các tab trình duyệt: landing `http://localhost:8090`, dashboard (tab 1), thư mục `evidence/`, `report/BaoCao_DoAn_BigData.pdf`, và một cửa sổ PowerShell ở thư mục repo.
4. Kiểm nhanh: `docker compose run --rm tools python loader/validate.py` phải ra 23/23 bảng và 19/19 số neo (chạy trước, không chạy lúc demo vì cần thời gian).
5. Chuẩn bị bản dự phòng: ảnh trong `evidence/`, PDF báo cáo, tài liệu này in sẵn.

## 4.2. Kịch bản từng bước

**Bảng 4.1.** Kịch bản demo (tổng khoảng 6 phút)

| Thời gian | Người | Thao tác | Lời nói gợi ý |
|---|---|---|---|
| 0:00–0:45 | Tường | Mở `docs/architecture.png` (hoặc Hình 4.1 trong PDF) | "Nhóm em làm Analytics → Presentation: nhận 23 bảng Parquet, nạp vào Neon bằng COPY, 22 view ngữ nghĩa, Superset 7 tab, và view gợi ý quyết định cho nhóm Decision." |
| 0:45–1:30 | Tường | Mở `evidence/load_run.log` và `validation.md` | "Lần nạp 13/11: 23 bảng, 505.592 dòng, 23/23 khớp số dòng; 19/19 số neo, ví dụ doanh thu 13.494.400,74 BRL khớp tới từng xu." |
| 1:30–2:00 | Hồng Phong | Mở landing `:8090`, cho xem biểu đồ phát lại và huy hiệu kiểm chứng; bấm "Mở dashboard" | "Landing là cửa vào, số liệu thật xuất từ Neon; nút này mở đúng dashboard." |
| 2:00–3:00 | Phi Long | Dashboard tab 1: chỉ 6 thẻ KPI, cột tháng 11/2017, đường % trễ vs review | "Tháng 11/2017 doanh thu đỉnh, tăng 52% nhưng trễ lên 14% và review rơi còn 3,95 — bán nhiều không đồng nghĩa vận hành tốt." |
| 3:00–3:45 | Phi Long | Thanh bộ lọc: Khoảng thời gian = 2018, Bang khách hàng = SP → Áp dụng | "Thẻ doanh thu từ 13,45 triệu còn 2,96 triệu; bảng KPI chỉ còn 8 tháng 2018. Bộ lọc chỉ gắn vào biểu đồ có cột tương ứng." Sau đó bấm "Xoá tất cả". |
| 3:45–4:30 | Phi Long | Tab 3 Địa lý (bản đồ Brazil, bubble) rồi tab 6 Giao trễ | "SP giao 8,7 ngày, review 4,21; các bang xa giao 15–19 ngày, review dưới 4. Trễ từ 4 ngày trở lên đa số khách cho 1 sao." |

| Thời gian | Người | Thao tác | Lời nói gợi ý |
|---|---|---|---|
| 4:30–5:15 | Phi Long | Tab 5 Người bán → bảng watchlist; tab 7 → bảng gợi ý quyết định | "153 người bán cần theo dõi, 75 mức đỏ kèm lý do và hành động; tab 7 là 7 luật mà nhóm Decision đọc bằng một câu SQL." |
| 5:15–6:00 | Hồng Phong | Mở `dashboard_load_latency.md`, `query_latency.md`, `scale_experiment.md` | "Truy vấn chỉ 1,25 ms phía server, 173 ms phía client do mạng; tab cold 3,83 s, có cache 1,51 s. Dữ liệu ×10 thì truy vấn thô 47 → 507 ms còn view tổng hợp giữ 1,1 ms." |
| 6:00–6:30 | Hồng Phong | Mở `qa_test_cases.md` | "41/41 ca kiểm thử tự động PASS; mọi lệnh trong README chạy được, người khác dựng lại bằng Docker Compose." |

Nếu thầy yêu cầu chứng minh tái lập tại chỗ: chạy `docker compose run --rm tools python loader/apply_sql.py` (tạo lại 22 view trong vài giây) rồi tải lại dashboard; không chạy nạp lại toàn bộ (mất ~5 phút).

## 4.3. Phương án dự phòng

**Bảng 4.2.** Xử lý sự cố khi demo

| Sự cố | Dấu hiệu | Cách xử lý |
|---|---|---|
| Docker Desktop không chạy / Superset không lên | `docker compose ps` không healthy, trang 8088 không mở | Chuyển sang ảnh: `evidence/dashboard_tab1…7_*.png`, `filter_*.png`, `watchlist_*.png`, `superset_giao_dien_dong_bo_*.png`; trình bày theo đúng thứ tự kịch bản trên |
| Neon chậm hoặc mất mạng | Biểu đồ quay lâu, báo lỗi kết nối | Nếu cache đã làm nóng thì dashboard vẫn hiện trong 10 phút; nếu không, dùng ảnh và nói rõ "dữ liệu trên Neon Singapore, đang mất kết nối" |
| Biểu đồ đầu tiên chậm 3–6 s | Spinner lâu ở lần mở đầu | Giải thích ngay: mỗi biểu đồ mở kết nối TLS mới (~1,1 s), không phải truy vấn chậm; mở lại tab sẽ ~1,5 s |
| Máy chiếu không mở được trình duyệt | — | Mở `report/BaoCao_DoAn_BigData.pdf`: Hình 4.1 (kiến trúc), chương 6 (số đo), chương 7 (ảnh dashboard) |
| Bộ lọc bị ghi đè sau khi chạy lại `build_dashboard.py` | Bộ lọc không như mong đợi | Tải lại trang (Superset 4.1 lưu trạng thái của tab trình duyệt cũ) |

## 4.4. Lưu ý khi trả lời

- Nói số tròn trước rồi số chính xác; luôn kèm nguồn ("trong `validation.md`", "Hình 4.1").
- Điều chưa làm thì nói thẳng là chưa làm và nêu hướng làm (streaming, autoscaling nhiều CU, đo riêng cold start compute, Neon Console screenshot).
- Phân vai trả lời theo mục 3.10: Tường – Neon, loader, tích hợp; Phi Long – view SQL, dashboard, bộ lọc; Hồng Phong – thực nghiệm, QA, báo cáo. Câu hỏi chung thì nhóm trưởng mở đầu, thành viên liên quan bổ sung.